



Rapport Travail 9 - Group Dih

REY Dorian

DURAND Eliot

BASILE Francesco-Pio

BOUREUX Nathan

Faculté des Sciences
Université de Montpellier

Avril 2026

Table des matières

1	Entraînement du modèle DistilBERT	3
1.1	Amélioration via Optuna	3
1.2	Test des modèles baseline et Optuna avec une répartition 50 / 50	4
1.3	Le modèle final	5
2	Agrégateur de vulnérabilités	6
3	Conteneurisation du backend	7
4	Déploiement du backend sur VPS	7
5	Rapport	7
6	Annexe	8

1 Entraînement du modèle DistilBERT

1.1 Amélioration via Optuna

Optuna est un framework open source permettant d'optimiser les hyperparamètres d'une manière plus logique que purement par hasard. Tous les résultats d'entraînement sont disponibles à la Fig. 1. Le code colab est disponible à la Fig. 2. Le meilleur modèle actuel est disponible à la Fig. 3.

Nous avons continué l'entraînement du modèle en utilisant Optuna. Ci-dessous les résultats d'une étude sur 12 essais :

Trial	Mean F1	Learning Rate	Dropout	Weight Decay	Status
0	0.8851	2.39e-5	0.1238	0.0936	Finished
1	0.8710	1.36e-5	0.2245	0.0932	Finished
2	0.8705	1.64e-5	0.3313	0.0953	Finished
3	0.8797	2.32e-5	0.1960	0.0481	Finished
4	0.8626	1.14e-5	0.2946	0.0106	Finished
5	0.8920	4.85e-5	0.3923	0.0432	Best
6	-	1.98e-5	0.3121	0.0759	Pruned
7	-	1.45e-5	0.1542	0.0331	Pruned
8	0.8918	4.53e-5	0.1306	0.0287	Finished
9	0.8902	3.19e-5	0.1905	0.0220	Finished
10	0.8877	4.78e-5	0.3699	0.0660	Finished
11	0.8902	4.80e-5	0.3910	0.0400	Finished

Tableau 1. – Résultats de l'optimisation des hyperparamètres via Optuna

Nous pouvons voir que Optuna détermine les meilleurs paramètres sur un jeu de données de 20 000 lignes le learning rate (% d'ajustement des poids lors de l'entraînement), le dropout (% de neurones désactivés aléatoirement pendant l'entraînement) et le weight decay (facteur de punition pendant l'entraînement pour éviter le sur-apprentissage).

Nous allons maintenant tester le modèle en appliquant les hyperparamètres donnés par Optuna. Ci-dessous le tableau des résultats :

Métrique	E1	E2	E3	E4	E5	Meilleur modèle pour le moment (baseline avec prétraitement)
Accuracy Train	0.8644	0.9110	0.9261	0.9379	0.9501	0.9244
Accuracy Val.	0.9030	0.9094	0.9103	0.9108	0.9091	0.9108
F1 Train	0.8556	0.9075	0.9238	0.9363	0.9491	0.9218
F1 Val.	0.8991	0.9069	0.9079	0.9084	0.9077	0.9084
Loss Train	2.8840	2.0228	1.6952	1.4189	1.1588	1.7231
Loss Val.	2.1734	2.0395	2.0522	2.1475	2.3042	2.0620

Tableau 2. – Entraînement avec les meilleurs hyperparamètres (LR: 4.85e-5, Drop: 0.39, WD: 0.04)

Dès l'époch 2, le modèle est très légèrement meilleur que le meilleur modèle pour l'instant (-0,02 de loss). Cependant il sur-apprend très vite (loss à 2,3 dès l'époch 5), ce qui est logique car notre learning rate est plus haut que les autres modèles, résultant d'un pic de performance plus précoce dans les epochs. En conclusion nous pouvons dire que le meilleur modèle actuellement est celui optimisé par Optuna à l'époch 2. Nous appellerons désormais ce modèle le « modèle Optuna ».

Nous avons fait ce qu'il fallait pour améliorer le modèle, nous pouvons encore tester d'autres approches. Par exemple, en s'inspirant du papier de recherche cité dans le précédent rapport, nous pourrions utiliser une répartition 50 / 50 entre les données d'entraînement et les données de tests. Cette méthode permet d'évaluer le modèle sur plus de données de test et être certain de sa robustesse. Nous pourrions par exemple le tester en 50 / 50 entre le meilleur modèle baseline et le meilleur modèle Optuna

Une fois cette tâche terminée, nous prendrons le meilleur modèle et l'entraînerons sur le dataset entier de 190 000 lignes, afin qu'il ait vu le plus d'exemples possible.

1.2 Test des modèles baseline et Optuna avec une répartition 50 / 50

Ci-dessous les résultats d'un 50 / 50 sur le modèle baseline :

Métrique	E1	E2	E3	E4	E5	E6	E7
Accuracy Train	0.8224	0.8883	0.9059	0.9183	0.9282	0.9370	0.9459
Accuracy Val.	0.8808	0.8958	0.9045	0.9052	0.9066	0.9055	0.9061
F1 Train	0.8076	0.8803	0.9016	0.9150	0.9257	0.9350	0.9443
F1 Val.	0.8700	0.8912	0.9007	0.9019	0.9038	0.9031	0.9041
Loss Train	3.5918	2.4756	2.1267	1.8652	1.6475	1.4508	1.2636
Loss Val.	2.5971	2.2939	2.1503	2.1381	2.1446	2.2288	2.2817

Tableau 3. – Entraînement sur le modèle baseline en répartition 50 / 50

Nous constatons pas de grandes différences entre une répartition 50 / 50 et 80 / 20, avec une val loss légèrement plus élevée pour le modèle 50 / 50 (+0,1). Ceci est normal car le modèle a moins d'exemples

pour s'entraîner. Nous confirmons donc que l'approche 80 / 20 semble être la meilleure, nous avons des résultats légèrement meilleurs mais à peu près le même sur-apprentissage, tout en ayant plus de données pour entraînement du modèle.

Ci-dessous les résultats d'un 50 / 50 sur le modèle Optuna :

Métrique	E1	E2	E3	E4	Meilleur modèle baseline 50 / 50
Accuracy Train	0.8439	0.9000	0.9189	0.9337	0.9183
Accuracy Val.	0.8878	0.9027	0.9061	0.9054	0.9052
F1 Train	0.8317	0.8949	0.9160	0.9315	0.9150
F1 Val.	0.8795	0.8994	0.9027	0.9030	0.9019
Loss Train	3.2279	2.2340	1.8421	1.5227	1.8652
Loss Val.	2.4646	2.1719	2.1263	2.2060	2.1381

Tableau 4. – Entraînement sur le modèle Optuna en répartition 50 / 50

Confirmation sur le fait que nos résultats sont similaires aux modèles 80 / 20, en plus de nous confirmer que le modèle Optuna est légèrement plus performant (-0,01 de val loss).

1.3 Le modèle final

Suite à nos expérimentations de ces deux dernières semaines, nous avons tous les éléments pour construire le meilleur modèle. En résumé :

- Source de données de qualité, avec CVE noté uniquement en 3.1 ou 3.0 validé par la NVD. 190 000 entrées
- Prétraitement avec abstraction des biais sémantiques pour maximiser la généralisation (numéro de versions, chemins de fichiers, etc)
- Répartition 80 / 20, soit 80% de données pour l'entraînement et 20% de données pour la validation
- Hyperparamètres optimisés par Optuna, avec ajustement du learning rate, du dropout et du weight decay

Nous allons ainsi entraîner un modèle sur cette base mais sur cette fois l'intégralité du dataset, soit 190 000 lignes au lieu de 50 000 choisies au hasard. Au vu de l'augmentation du learning rate et de la taille du dataset, notre modèle devrait être le meilleur autour de 2 voire 3 epoch, avant de sur-apprendre.

Le but est d'ici gagner quelques % d'accuracy et stabiliser la loss, selon la loi des rendements décroissants, à partir de 90%, gagner un seul pourcent en plus est très demandant en ressource et en temps.

Ci-dessous les résultats de notre modèle final, que nous appellerons le modèle « full » :

Métrique	E1	E2	E3	E4	Meilleur modèle actuel
Accuracy Train	0.8957	0.9207	0.9313	0.9411	0.9110
Accuracy Val.	0.9162	0.9209	0.9230	0.9226	0.9094
F1 Train	0.8915	0.9185	0.9297	0.9400	0.9075
F1 Val.	0.9140	0.9193	0.9214	0.9215	0.9069
Loss Train	2.2957	1.7798	1.5413	1.3206	2.0228
Loss Val.	1.8644	1.7759	1.7370	1.7745	2.0395

Tableau 5. – Entraînement sur le modèle full

Les résultats sont très bons, la loss est descendue de presque 0,3, l'accuracy et la F1 ont gagné environ 1,5. Ce modèle est de loin le meilleur, il surpasse les baselines académiques classiques voire certains papiers de recherches. Nous utiliserons ce modèle dans notre pipeline final.

Ci-dessous nous reprenons les 9 CVE testées la semaine dernière comparées avec notre nouveau modèle :

CVE ID	Score NVD (Réal)	Score Baseline	Score Full Model
CVE-2024-22288	7.1	6.1	6.1
CVE-2024-27073	5.5	5.5	5.5
CVE-2024-46591	7.5	7.5	7.5
CVE-2025-5799	8.8	9.8	8.8
CVE-2025-4394	6.8	6.1	6.1
CVE-2025-10442	9.8	9.8	9.8
CVE-2026-23072	5.5	5.5	5.5
CVE-2026-0007	8.6	7.8	7.8
CVE-2026-1663	4.3	4.3	4.3

Tableau 6. – Comparaison des scores CVSS entre les données réelles (NVD), le modèle baseline (temporel) et le modèle final (full dataset)

Les résultats sont imparfaits, mais ils restent cohérents, les modèles ont les mêmes prédictions à une CVE près, la 2025-5799 (en gras) où le modèle full s'est amélioré.

2 Agrégateur de vulnérabilités

Conformément aux objectifs définis la semaine dernière, nous avons corrigé la plupart des problèmes de l'agrégateur, notamment celui du décalage entre les noms de paquets Maven et les nomenclatures des différentes bases de données.

Les principaux problèmes provenaient de l'OSV et de Github Advisory, qui ont besoin du nom exact du distributeur du package. Ces sources étaient tout simplement ignorées si le logiciel ne trouvait pas le distributeur dans notre dictionnaire de correspondances entre les noms des packages et leur noms complets (avec distributeur), ou s'il ne pouvait pas le déduire simplement (ex : org.junit.jupiter).

Notre solution pour pallier ce problème est de demander directement à Maven. C'est-à-dire faire une requête de recherche du package pour récupérer son distributeur. Grâce à cette solution, nous avons pu supprimer les dictionnaires de correspondances, la transformation de nom de package à un nom acceptable par l'OSV et Github est donc automatique et les requêtes à ces bases de données fonctionnent correctement.

3 Conteneurisation du backend

Dans le but d'intégrer une dimension industrielle au projet, il est important d'automatiser le déploiement. Une première partie de ce travail a été faite avec le backend. Il suffit simplement d'exécuter la commande suivante : `docker compose up --build`.

Le fichier `docker-compose.up` télécharge les dépendances ainsi que le modèle d'IA s'il n'est pas déjà en local. La base de données est créée à partir de l'image officielle de MySQL et inclut de la persistance grâce à un volume.

4 Déploiement du backend sur VPS

Afin de rendre le projet flexible, même si une version 100% locale est possible, une version conteneurisée du backend a été déployée sur un VPS. Il suffit de remplacer dans le frontend l'IP du `BACKEND_URL` dans le `.env` par celui du VPS et n'avoir qu'à lancer ce dernier. L'IP est la suivante : 51.83.76.101:8000 (de la latence à prévoir car le modèle tourne sur un CPU, donc il est plus long qu'un PC avec un GPU).

5 Rapport

Une première version du rapport est disponible en pièce jointe du mail.

6 Annexe

1. https://docs.google.com/document/d/11eCs1nqLkFaR1OKISUttD2aPfVNNbwsUF24GuN8RB_w/edit?usp=sharing
2. <https://urlix.me/cvss-colab-code>
3. <https://urlix.me/cvss-models>